Laboratorio 8

Antony David Saz

Teoria de la Computacion

Problema 1

Parte a: Complejidad

El programa analizado es:

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}
```

El primer ciclo inicia en $n/2$ y termina en n, por lo que ejecuta aproximadamente:

$$n - \frac{n}{2} + 1 = \frac{n}{2} + 1$$

Esto pertenece a $\Theta(n)$.
El segundo ciclo cumple la condicion:

$$j + \frac{n}{2} \leq n$$

Esto es equivalente a:

$$j \leq \frac{n}{2}$$

Por lo tanto, el segundo ciclo tambien ejecuta aproximadamente $\frac{n}{2}$ veces, es decir, $\Theta(n)$.
El tercer ciclo multiplica k por 2 en cada iteracion:

$$1, 2, 4, 8, \ldots, n$$

La cantidad de iteraciones es:

$$\lfloor \log_2(n) \rfloor + 1$$

Por lo tanto, el tercer ciclo pertenece a $\Theta(\log n)$.
Entonces:

$$T(n) = \Theta(n) \cdot \Theta(n) \cdot \Theta(\log n)$$

$$T(n) = \Theta(n^2 \log n)$$

Complejidad en notacion Big-Oh:

$$\boxed{O(n^2 \log n)}$$

Parte b: Profiling

Para entradas muy grandes se uso profiling calibrado, porque ejecutar todos los bucles completos para $n = 1,000,000$ no termina en tiempo razonable.

n	Iteraciones logicas	Tiempo (s)	Modo
1	2	0.000001	medido
10	120	0.000014	medido
100	17850	0.001919	medido
1000	2505000	0.405864	medido
10000	350070000	56.719	estimado
100000	42500850000	6886.05	estimado
1000000	5000010000000	810109.21	estimado

Cuadro 1: Resultados de profiling para el Problema 1

Subir la grafica $\texttt{problema}_1.png a Overleaf para mostrarla aqui.$
`Grafica de input vs. tiempo para el Problema 1`

Figura 1: Grafica de input vs. tiempo para el Problema 1

Problema 2

Parte a: Complejidad

El programa analizado es:

```
void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            break;
        }
    }
}
```

Si $n \leq 1$, el programa termina inmediatamente, por lo que toma tiempo constante.

Para $n > 1$, el ciclo externo se ejecuta n veces. Aunque el ciclo interno parece recorrer desde 1 hasta n, contiene una instruccion `break`. Esto significa que el ciclo interno solo se ejecuta una vez por cada iteracion del ciclo externo.

Entonces:

$$T(n) = n \cdot 1$$

$$T(n) = \Theta(n)$$

Complejidad en notacion Big-Oh:

$$\boxed{O(n)}$$

Parte b: Profiling

n	Iteraciones logicas	Tiempo (s)	Modo
1	0	0.000000500	medido
10	10	0.000002	medido
100	100	0.000016	medido
1000	1000	0.000214	medido
10000	10000	0.002143	medido
100000	100000	0.019355	medido
1000000	1000000	0.184201	medido

Cuadro 2: Resultados de profiling para el Problema 2

Subir la grafica `problema2.png` a Overleaf para mostrarla aqui.
`Grafica de input vs. tiempo para el Problema 2`

Figura 2: Grafica de input vs. tiempo para el Problema 2

Problema 3

Parte a: Complejidad

El programa analizado es:

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
        }
    }
}
```

El ciclo externo se ejecuta desde 1 hasta $n/3$, por lo que realiza aproximadamente:

$$\frac{n}{3}$$

Esto pertenece a $\Theta(n)$.

El ciclo interno empieza en 1 y aumenta de 4 en 4 hasta llegar a n. Por lo tanto, realiza aproximadamente:

$$\frac{n}{4}$$

Esto tambien pertenece a $\Theta(n)$.

Entonces:

$$T(n) = \Theta(n) \cdot \Theta(n)$$

$$T(n) = \Theta(n^2)$$

Complejidad en notacion Big-Oh:

$$\boxed{O(n^2)}$$

Parte b: Profiling

Para entradas muy grandes se uso profiling calibrado, porque ejecutar todos los bucles completos para $n = 1,000,000$ no termina en tiempo razonable.

n	Iteraciones logicas	Tiempo (s)	Modo
1	0	0.000000700	medido
10	9	0.000002	medido
100	825	0.000115	medido
1000	83250	0.012947	medido
10000	8332500	1.237	medido
100000	833325000	123.661	estimado
1000000	83333250000	12366.25	estimado

Cuadro 3: Resultados de profiling para el Problema 3

Subir la grafica `problema3.pngaOverleafparamostrarlaaqui.`
`Grafica de input vs. tiempo para el Problema 3`

Figura 3: Grafica de input vs. tiempo para el Problema 3

Problema 4

Linear Search

En busqueda lineal se revisa cada elemento desde el inicio hasta encontrar el objetivo o hasta terminar el arreglo.

Mejor caso:

$$\Theta(1)$$

Ocurre cuando el elemento buscado esta en la primera posicion. Solo se necesita una comparacion.

Caso promedio:

$$\Theta(n)$$

Si el elemento tiene la misma probabilidad de estar en cualquier posicion, el numero esperado de comparaciones es:

$$\frac{1+2+3+\cdots+n}{n} = \frac{n+1}{2}$$

Como $\frac{n+1}{2}$ crece linealmente, el caso promedio es $\Theta(n)$.

Peor caso:

$$\Theta(n)$$

Ocurre cuando el elemento esta en la ultima posicion o no esta en el arreglo. En ambos casos se revisan hasta n elementos.

Binary Search

Binary Search requiere que el arreglo este ordenado. En cada paso compara con el elemento del centro y elimina la mitad del arreglo restante.

Mejor caso:

$$\Theta(1)$$

Ocurre cuando el elemento buscado esta justo en la posicion central.

Caso promedio:

$$\Theta(\log n)$$

En cada iteracion el problema se reduce a la mitad. La recurrencia es:

$$T(n) = T(n/2) + c$$

Despues de aproximadamente $\log_2(n)$ divisiones se llega a un solo elemento, por lo que el crecimiento es logaritmico.

Peor caso:

$$\Theta(\log n)$$

Ocurre cuando el elemento esta en uno de los ultimos subarreglos revisados o no se encuentra. Aun asi, el arreglo se parte a la mitad en cada paso.

Quick Sort

Quick Sort elige un pivote, particiona el arreglo y luego ordena recursivamente las dos partes.

Mejor caso:

$$\Theta(n \log n)$$

Ocurre cuando el pivote divide el arreglo en dos partes de tamanos similares. La recurrencia es:

$$T(n) = 2T(n/2) + cn$$

Por el teorema maestro:

$$T(n) = \Theta(n \log n)$$

Caso promedio:

$$\Theta(n \log n)$$

En promedio, las particiones no son perfectamente iguales, pero suelen estar suficientemente balanceadas. Por eso el costo esperado sigue siendo $\Theta(n \log n)$.

Peor caso:

$$\Theta(n^2)$$

Ocurre cuando el pivote siempre queda como el menor o el mayor elemento. En ese caso una particion queda vacia y la otra queda con $n - 1$ elementos:

$$T(n) = T(n-1) + cn$$

Al sumar los costos:

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n(n+1)}{2}$$

Por lo tanto, el peor caso es:

$$\Theta(n^2)$$

Problema 5

Inciso a

Enunciado:

$$\text{Si } f(n) = \Theta(g(n)) \text{ y } g(n) = \Theta(h(n)), \text{ entonces } h(n) = \Theta(f(n)).$$

Respuesta: Verdadero.

Por definicion, si $f(n) = \Theta(g(n))$, entonces existen constantes positivas c_1, c_2 y n_0 tales que:

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

para todo $n \geq n_0$.

Tambien, si $g(n) = \Theta(h(n))$, existen constantes positivas c_3, c_4 y n_1 tales que:

$$c_3 h(n) \leq g(n) \leq c_4 h(n)$$

para todo $n \geq n_1$.

Esto significa que $f(n)$, $g(n)$ y $h(n)$ crecen al mismo orden asintotico. Por transitividad de Θ:

$$f(n) = \Theta(h(n))$$

Como Θ es una relacion simetrica, tambien se cumple:

$$h(n) = \Theta(f(n))$$

Por lo tanto, el enunciado es verdadero.

Inciso b

Enunciado:

$$\text{Si } f(n) = O(g(n)) \text{ y } g(n) = O(h(n)), \text{ entonces } h(n) = \Omega(f(n)).$$

Respuesta: Verdadero.
Si $f(n) = O(g(n))$, entonces existe una constante positiva c_1 tal que:

$$f(n) \leq c_1 g(n)$$

para valores suficientemente grandes de n.
Si $g(n) = O(h(n))$, entonces existe una constante positiva c_2 tal que:

$$g(n) \leq c_2 h(n)$$

para valores suficientemente grandes de n.
Sustituyendo:

$$f(n) \leq c_1 c_2 h(n)$$

Esto se puede reordenar como:

$$h(n) \geq \frac{1}{c_1 c_2} f(n)$$

Por definicion:

$$h(n) = \Omega(f(n))$$

Por lo tanto, el enunciado es verdadero.

Inciso c

Enunciado:

$$f(n) = \Theta(n^2)$$

donde $f(n)$ esta definido como el tiempo de ejecucion del siguiente programa Python:

```
def A(n):
    atup = tuple(range(0, n))
    S = set()
    for i in range(0, n):
        for j in range(i + 1, n):
            S.add(atup[i:j])
```

Respuesta: Falso.

A primera vista el programa parece ser $\Theta(n^2)$ porque tiene dos ciclos anidados:

```
for i in range(0, n):
    for j in range(i + 1, n):
```

La cantidad de pares (i, j) es:

$$\frac{n(n-1)}{2}$$

Esto es $\Theta(n^2)$.

Sin embargo, dentro del ciclo se ejecuta:

```
atup[i:j]
```

En Python, hacer un slice de una tupla crea una nueva tupla. El costo de crear `atup[i:j]` depende del tamano del slice, es decir, cuesta:

$$\Theta(j - i)$$

Ademas, al insertar la tupla en el set, Python debe calcular su hash, lo cual tambien depende de la longitud de la tupla creada.

Por lo tanto, el costo total no solo cuenta cuantas veces se ejecuta el ciclo, sino tambien el costo de copiar los slices:

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j - i)$$

Esta suma crece como:

$$\Theta(n^3)$$

Por lo tanto, el tiempo de ejecucion real del programa es:

$$\Theta(n^3)$$

El enunciado dice que es $\Theta(n^2)$, por lo tanto es falso.